

SUPPLY CHAIN MANAGEMENT

Using the IT industry's best kept secrets (secret sauce) to estimate cost

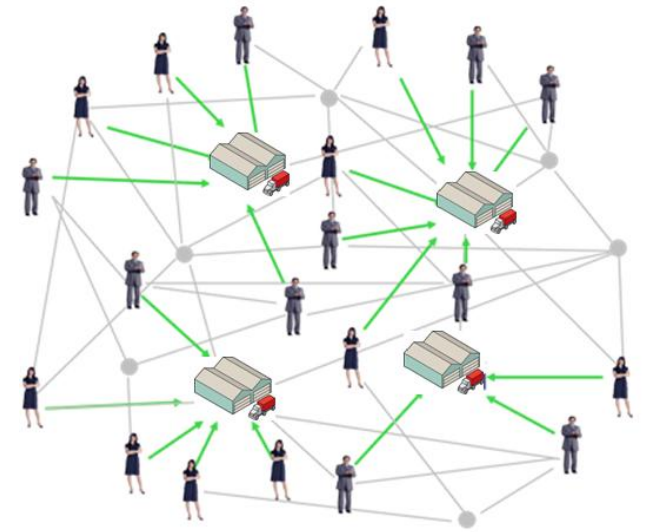


Supply-Chain Management

Project – III: post-mortem

➤ **Given:** a set of items possible facility locations
a set customer with certain demand
an upper bound on the number of vehicles

➤ **Find:**



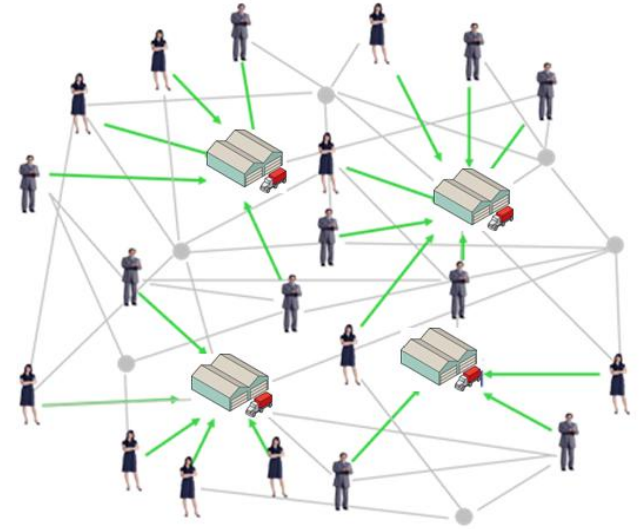
Supply-Chain Management

Project – III: post-mortem

➤ **Given:** a set of items possible facility locations
a set customer with certain demand
an upper bound on the number of vehicles

➤ **Find:** a lower bound on
opening facility locations
customer allocations
respect vehicle capacity limits

minimize opening + service + driving



Supply-Chain Management

Project – III: post-mortem

➤ **Decision variables:**

- isOpenFacility[f]
- isUsedFacilityVehicle[f][v]
- isServedCustomerFacilityVehicle[c][f][v]

Supply-Chain Management

Project – III: post-mortem

➤ **Decision variables:** $\text{isOpenFacility}[f]$
 $\text{isUsedFacilityVehicle}[f][v]$
 $\text{isServedCustomerFacilityVehicle}[c][f][v]$

➤ **Constraints:**

vehicle cannot work if facility is not open

$$\text{isUsedFV} \leq \text{isOpenF} \quad \forall f, v$$

customer cannot be served if vehicle is not used

$$\text{isServedCFV} \leq \text{isUsedFV} \quad \forall c, f, v$$

Supply-Chain Management

Project – III: post-mortem

➤ **Decision variables:**

isOpenFacility[f]

isUsedFacilityVehicle[f][v]

isServedCustomerFacilityVehicle[c][f][v]

➤ **Constraints:**

served by 1 facility

$$\sum_{f,v} \text{isServedCFV} = 1 \quad \forall c$$

driving limits

$$\sum_c \text{distCF} \times \text{isServedCFV} \leq \text{lim} \times \text{isUsedFV} \quad \forall f, v$$

capacity

$$\sum_{c,v} \text{demandC} \times \text{isServedCFV} \leq \text{capF} \times \text{isOpenF} \quad \forall f$$

Supply-Chain Management

Project – III: post-mortem

➤ **Decision variables:** `isOpenFacility[f]`
 `isUsedFacilityVehicle[f][v]`
 `isServedCustomerFacilityVehicle[c][f][v]`

Challenges and Optimizations

1. Designing the model
 - a. Coming up with the decision variables I wanted to use was, by far, the hardest part of this project. The majority of my time on this project (~10 hours) was spent over the course of a few days, burning through endless pages of scrap paper as I tried to plot out the best strategy for implementing the variables. I considered models that used edge matrices, three different sets of variables, and even 3D

Supply-Chain Management

Project – III: post-mortem

➤ **Decision variables:** `isOpenFacility[f]`
 `isUsedFacilityVehicle[f][v]`
 `isServedCustomerFacilityVehicle[c][f][v]`

numFacilities. Eventually, what helped the most was really wrapping my head around the fact that this was a linear programming problem, something I did technically know but hadn't fully understood what that meant in terms of implications on my model. Knowing that the solution would have some weird values allowed me to plot more model options, and eventually for the main

Supply-Chain Management

Project – III: post-mortem

➤ **Decision variables:** isOpenFacility[f]
 isUsedFacilityVehicle[f][v]
 isServedCustomerFacilityVehicle[c][f][v]

The ultimate goal of the problem is to decide:

1. which facilities to open (the buildings in Figure 1).
2. which vehicles to deploy in each facility.
3. which facility and vehicle to use when serving each customer.

Direct mapping from the project handout!

Supply-Chain Management

Project – III: post-mortem

➤ **Decision variables:** `isOpenFacility[f]`
 `isUsedFacilityVehicle[f][v]`
 `isServedCustomerFacilityVehicle[c][f][v]`

By nature, this problem is a discrete problem, so turning it into a linear programming exercise was distinctly unsatisfying. There are several variables that require some sort of discrete variable: whether a customer is being served by a given facility, whether a factory is open, and the number of cars deployed by a given factory. Each of these discrete variables is associated with a cost that does not scale; either all of the cost was applied (i.e. you must pay the entirety of the factory cost if you use a factory) or none of it was, making modeling an objective function impossible with linear variables. At first, I tried to work around these constraints by trying to “hack in” discrete jumps using highly scaled sigmoid functions, but since these functions are not linear and thus incompatible with LP solvers. I ended up solving the problem like a boolean IP problem but using continuous bounds for the boolean variables instead of discrete ones. This would provide a good lower bound.

Supply-Chain Management

Project – III: post-mortem

Overall, this project was quite enjoyable. It was interesting to see how to create a linear relaxation, and use it to extract meaningful information about a problem without strictly modeling the exact problem (didn't model using integer variables).

➤ Constraints:

Overall, this process made me uncomfortable. We would be satisfying constraints by sending 0.5 trucks to serve 0.2 of a customer by traveling 0.3 of the round trip from a facility paying 0.8 of its operational cost, which makes no conceptual sense. However, as a lower bound with relaxed constraints, it makes sense to use this function. I am, however, slightly disappointed that my favorite screen name was used on a relatively simple project, and the results that come back have no meaning since not all of them are integers.

c, v

Most of the time of this project was trying to figure out how to represent the constraints in a linear program. The break through was when I realized I try to model the underlying integer program, and then just replace the integers with continuous variables. After I did that, my first intuition seemed to match the leaderboards.

I found that it required me to think very differently than when using the constraint solver. For example, I wanted to make a single assignment variable per customer, that would store the index of the facility that that customer is assigned to. However, using a continuous variable as an index is impossible, so I needed to make a bunch of “boolean” variables per customer, and then use multiplication to achieve what I would have done with filtering and implication. I found this to be a new and interesting way of approaching modeling in general.

For the most part, the project was fairly straightforward, although there were a few nuances in the specification of the (relaxed) problem that influenced how I structured the decision variables and constraints.

The most notable decision to make was whether or not to explicitly track vehicle allocations to customers. Doing so would give us stricter bounds on the problem and would also provide more

1.2 Constraint Model

These constraints were very straightforward.

When we first wrote up our constraints, we forgot some small details (like multiplying capacity by a facility's status). This led us to having really bad lower bounds, so we tried to remedy this issue by adding a bunch of extraneous concerns and trying to model the problem in a completely different way. In hindsight, trying to do unit tests on smaller examples would have helped us figure out the details we were missing. However, after talking with the professor we were able to deduce that our original intuition was correct, and we probably mis-coded a small detail. We reverted back to our original structure and found our error. In the future, we will be writing more unit tests and trusting our original intuitions.

After doing this, I noticed that I was receiving an unbelievable result - a staggering 0 minimum cost, what a spectacular win for everybody! Anyways, I was obviously doing something wrong and then realized I'd forgotten to add a constraint handling the fact that a person must be serviced by one facility. I added a constraint stating that the sum of all the facilities that a person has been

2 Relaxing from IP to LP

It wasn't trivial to relax the integer programming problem to a linear programming problem. To accomplish this, we relax the constraint that a customer is

This assignment was quite straightforward compared to the previous assignments. There was essentially no model finicking needed to determine the optimal (relaxed) assignment very quickly. Perhaps this is just due to the relatively easy nature of the linear programming problem (the only problem we've looked at so far that isn't NP-Hard).

between 0 and 1 as opposed to having two possible values. Due to that, I originally approached the variable creation, as well as the rest of the problem, as if it were basically an integer programming problem.

the hope of small fixing bugs. But we enjoyed the modeling portion and the challenge of creating a linear relaxation – it really tested our abilities to think of how our constraints applied to the real world.

Decision Variables

This part was honestly the hardest part, since I had to figure out how to make the intuitively discrete variables into continuous variables for the linear program. The key was to realize that we

We had some initial struggles working with continuous variables in an inherently discrete problem, specifically in the facility opening cost. We first tried summing the percentage of demand fulfilled

if a facility is servicing somebody, it should be listed as open). This caused my results to match the majority of results in the leaderboard. The way I remodeled the constraint made sense to me in terms of the code, but because it was giving me results that were generally close but different from the others' results, I ultimately caved to the leaderboard peer pressure and kept my code with that constraint commented out - I'm unsure, however, if the constraint was correct/should have remained there. Maybe I should have believed in my constraints or maybe everybody else was right, I'll never know (I will probably know in about a week).

Results

First, I forgot to add the proportionality of the capacity and openness of facility in the constraint (2), so my total cost were so low. I was happy with the result and submitted, because I saw that the 3 other groups on the leader board had similar results. But then next day I checked the leader board again and saw that one of the groups changed their submission and now they have much greater costs for all the examples. That made me think if I forget to consider something and figured out the situation. After that I saw other groups also changed their submissions.

there some range within which our answers were expected to be. This latter part was particularly hard for us since we finished the project super early on due to Spring Break and had no answers on the leaderboard to compare with. We found some solace when other students started getting similar values to us. Lot of them having exactly the same value. This indicated to us that we were

I didn't end up needing to tweak this design too much. I started a little late, so by the time I submitted to Gradescope, there were plenty of results to compare to, and my numbers were pretty consistent with what others in the class were getting.

We inferred the correctness of our model by comparing our results with other solutions on the leaderboard. We shared the same scores with several other submissions per problem, and hence concluded that our model was most likely correct.

Because I couldn't really write a checker for this project, the leaderboard was super helpful in checking whether I was on the right track. I started with very bounds lower than everyone else's, and that told me I was probably either missing a constraint or adding terms that were too small in value to my objective function (and that I might need to scale them up by some factor). I kept working until my results matched with the majority of the leaderboard.

rather, we are ranked by our interpretation of the problem. A higher total lower bound score could either indicate a better LP because of tighter constraints, but it could also just be an incorrect interpretation of the problem.

One of the most interesting aspects of this project was the shift in perspective when designing constraints. Initially, I focused on adding constraints that might improve runtime, but this often led to redundant rules that did not actually reduce the feasible space. Over time, I shifted toward thinking purely in terms of tightening the search space, regardless of computational overhead. This mindset was more effective and conceptually cleaner, as it required reasoning about the geometry of the feasible region rather than the internal mechanics of the solver.

Verification

Because we don't know the optimal integer solution to each instance, it's hard to know whether my solution to the linear relaxation is actually correct. So, to verify the correctness of my model, I checked two properties on toy datasets where the optimal integer solution is indeed known: (1) the IP solver should produce the true optimal integer solution, and (2) the LP relaxation should always provide a valid lower bound ($LP \leq IP$). Indeed, the optimum is definitionally the lowest cost - by hand calculating the optimal cost in the integer formulation of the problem, and comparing it to the output of that same input with the linear relaxation, we can see that the

Instance	Baseline	With Cuts	Improv.	Instance	Baseline	With Cuts	Improv.
C1_40_150	2693	2959	+9.9%	R1_100_50	1586	1652	+4.2%
C1_50_100	2047	2252	+10.0%	R1_40_100	2681	3041	+13.4%
C10_40_100	14557	14898	+2.3%	R1_40_50	2041	2269	+11.2%
C10_50_300	16485	17299	+4.9%	R18_100_100	10500	10611	+1.1%
C18_100_150	10496	10596	+1.0%	R18_50_300	14251	14974	+5.1%
C18_40_80	10843	11238	+3.6%				

Table 1: Comparison of baseline LP vs LP with tightening cuts. Average improvement: +4.1%.

Tightening Cuts

The basic vehicle constraint only says total distance must fit, ignoring that customers can't always share vehicles. We add two cuts:

First Cut: Floor-based Vehicle Bound. At most $\lfloor L/d \rfloor$ customers with distance d can share one vehicle, so each needs $\geq 1/\lfloor L/d \rfloor$ vehicles. This is tighter than d/L when $d > L/2$ (e.g., $d = 0.6L$ gives 1.0 vs 0.6).

$$v_f \geq \sum_c \frac{1}{\lfloor L/d_{c,f} \rfloor} \cdot x_{c,f} \quad \forall f$$

Second Cut: Pairwise Incompatibility. If $d_{c_1,f} + d_{c_2,f} > L$, customers c_1, c_2 can't share a vehicle:

$$v_f \geq x_{c_1,f} + x_{c_2,f} \quad \forall f, \forall c_1, c_2 \text{ where } d_{c_1,f} + d_{c_2,f} > L$$

This adds $O(C^2 \times F)$ constraints, but the solver handles them efficiently.

1.2 Preprocessing (Variable Bound Tightening)

Before adding any constraints we tighten the variable bounds analytically to reduce the solver's search space.

Infeasible assignments. If the round-trip distance between customer c and facility f exceeds the truck distance limit D , a single truck cannot complete the trip; hence we fix $\bar{x}_{c,f} = 0$ for such pairs and $\bar{x}_{c,f} = 1$ otherwise:

$$\bar{x}_{c,f} = \begin{cases} 1 & \text{if } d_{c,f} \leq D, \\ 0 & \text{otherwise.} \end{cases}$$

Vehicle upper bounds. An aggregate upper bound on vehicles at each facility is computed as the sum of all round-trip distances in that facility's column divided by the distance limit, rounded up and capped by the maximum vehicle count V :

$$\bar{n}_f = \min\left(V, \left\lceil \frac{\sum_c d_{c,f}}{D} \right\rceil\right).$$

This is a conservative (loose) bound, but it eliminates clearly unreachable vehicle counts at no modelling cost.

Another takeaway is that **preprocessing matters**. Just as in project 2, we want to **give the solver knowledge** it would have to discover through search. By adding the global constraint that open facilities must cover all the demand, which is implied by the demand and capacity constraints. However, this allows the solver to consider facility selection without having to iterate through individual constraints.

Symmetry breaking. Vehicles within each facility are indexed in non-increasing order of usage to reduce symmetry: $t_{f,k} \geq t_{f,k+1} \forall f \in F, k \in \{0, \dots, K_{\max} - 2\}$.

Clique cuts. A greedy procedure finds maximal cliques Q of mutually incompatible customers (by distance or demand), and clique-cardinality constraints are added: $\sum_{c \in Q} y_{c,f} \leq \sum_k t_{f,k}$ (distance clique) and $\sum_{c \in Q} y_{c,f} \leq f_f$ (capacity clique).

Prefix cuts. For growing prefixes P of customers sorted by descending distance (or demand), if at most m customers from the prefix can fit in one bin (truck or facility capacity): $\sum_{c \in P} y_{c,f} \leq m \cdot \sum_k t_{f,k}$ (distance) and $\sum_{c \in P} y_{c,f} \leq m \cdot f_f$ (capacity).



BUT THEN I realized this isn't really linear! I was multiplying decision variables by other decision variables, and that isn't really covered by the model as discussed in class. So I had to simplify things real quick. Instead of multiplying a facility's output by its degree of construction (non-linear) I could multiply its *capacity* by its degree of construction (linear!). Similarly, I could

Supply-Chain Management

Project – III: post-mortem

definitely a lower time commitment than the previous ones.

implementing it, and writing up the report. I really enjoyed working on it, and found that after I spent a while on modeling the assignment, the programming was relatively simple.

This project was very interesting, because it required more thought and planning than anything else. The lines of code I wrote were extremely simple and extremely quick, but thinking about the most efficient data structures for the constraints was thought-provoking. It was a bit difficult for me to wrap my head around the idea of turning a problem with discrete variables into continuous variables, but I ended up writing the constraints as if the model used discrete variables then made

Supply-Chain Management

Making the model:

I have attached a one page pdf below where I have outlined the process I undertook to devise the model needed for the problem. All that was required from the project handout was about 12 lines for requirements/constraints and about 3 lines for the objective. I read the handout twice and then referred to only these 15 lines while doing the project.

How is 15 lines sufficient?

After reading the handout, my observations were simple -

1. Meet customer demand - **constraint**
2. Enforce the capacity of the facility and the distance limit of the trucks - **constraint**
3. Minimize the cost - **objective function**

c, v

Implementation

I think I have developed a habit of search and inference even while reading the handouts for this class. When I was looking through the requirements (searching), I was constantly trying to see if it was a cost or a constraint (inference). For example, the truck's distance limit is a constraint while the cost of opening a facility is cost. After deciding what belonged to the constraint section and to the cost section, it was easy to come up with the equations.

We (accidentally) learned how much slower integer programming is by encoding our model using integer programming first. We also appreciate now *why* linear programming provides a lower bound.

INSTANCE	C10_40_100	C10_50_300	C1_40_150	C1_50_100	C18_100_150	C18_40_80
IP RESULT	15615	17989	3140	2293	11436	11596
IP TIME	406.49	347.01	19.79	17.39	1139.29	240385.00
LP RESULT	14536	16466	2474	1891	10729	10852
LP TIME	0.18	0.17	0.16	0.16	0.21	0.20
INT GAP	1.07	1.09	1.27	1.21	1.06	1.07
INSTANCE	R1_100_50	R1_40_100	R1_40_50	R18_100_100	R18_50_300	:)
IP RESULT	1776	3056	2334	11106	15208	
IP TIME	20.49	23.45	8.00	13478.09	2351.60	
LP RESULT	1524	2574	1964	10595	14316	
LP TIME	0.18	0.16	0.16	0.20	0.20	
INT GAP	1.17	1.19	1.19	1.05	1.06	

Supply-Chain Management

Project – III: post-mortem

- How to test
correctness?
- How to make it
more realistic?
- What are the
simplifying
assumptions?

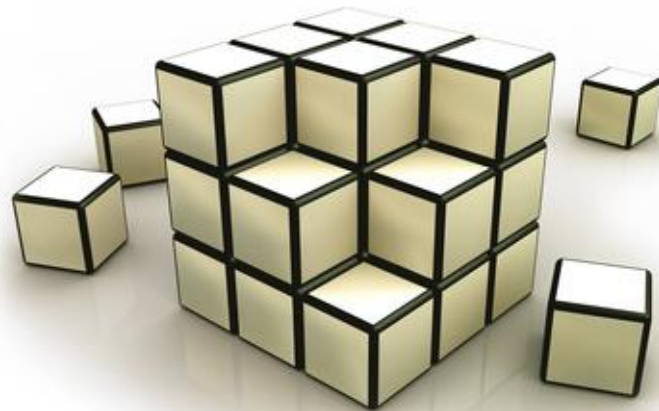
PROJECT - III: Supply Chain Management							
C10_40_100.scn	C10_50_300.scn	C18_100_150.scn	C18_40_80.scn	C1_40_150.scn	C1_50_100.scn	R18_100_100.scn	R18
14536.0 [0.15]	16439.0 [0.16]	10482.0 [0.19]	10828.0 [0.21]	2474.0 [0.10]	1890.0 [0.16]	10471.0 [0.18]	
14536.0 [0.22]	16439.0 [0.25]	10482.0 [0.36]	10828.0 [0.48]	2474.0 [0.15]	1890.0 [0.14]	10471.0 [0.53]	
14536.0 [0.12]	16439.0 [0.11]	10482.0 [0.15]	10828.0 [0.14]	2474.0 [0.11]	1890.0 [0.10]	10471.0 [0.13]	
14536.0 [0.27]	16439.0 [0.17]	10482.0 [0.22]	10828.0 [0.33]	2474.0 [0.13]	1890.0 [0.16]	10471.0 [0.22]	
14557.0 [1.38]	16505.0 [1.19]	10564.0 [3.26]	10848.0 [3.08]	2693.0 [0.34]	2047.0 [0.36]	10543.0 [3.21]	
3576.0 [0.73]	3776.0 [0.62]	3149.0 [1.41]	3079.0 [1.49]	1110.0 [0.28]	1060.0 [0.31]	3099.0 [1.04]	
14536.0 [1.04]	16439.0 [0.15]	10482.0 [0.14]	10828.0 [0.14]	2474.0 [0.12]	1890.0 [0.12]	10471.0 [0.14]	
14536.0 [0.24]	16439.0 [0.28]	10482.0 [0.26]	10828.0 [0.31]	2474.0 [0.25]	1890.0 [0.19]	10471.0 [0.23]	
14536.0 [0.08]	16439.0 [0.08]	10482.0 [0.09]	10828.0 [0.09]	2474.0 [0.07]	1890.0 [0.07]	10471.0 [0.08]	
5642.0 [0.23]	8494.0 [0.23]	4544.0 [0.26]	5043.0 [0.26]	2519.0 [0.19]	1817.0 [0.18]	4190.0 [0.27]	

Supply-Chain Management

Project – III: post-mortem

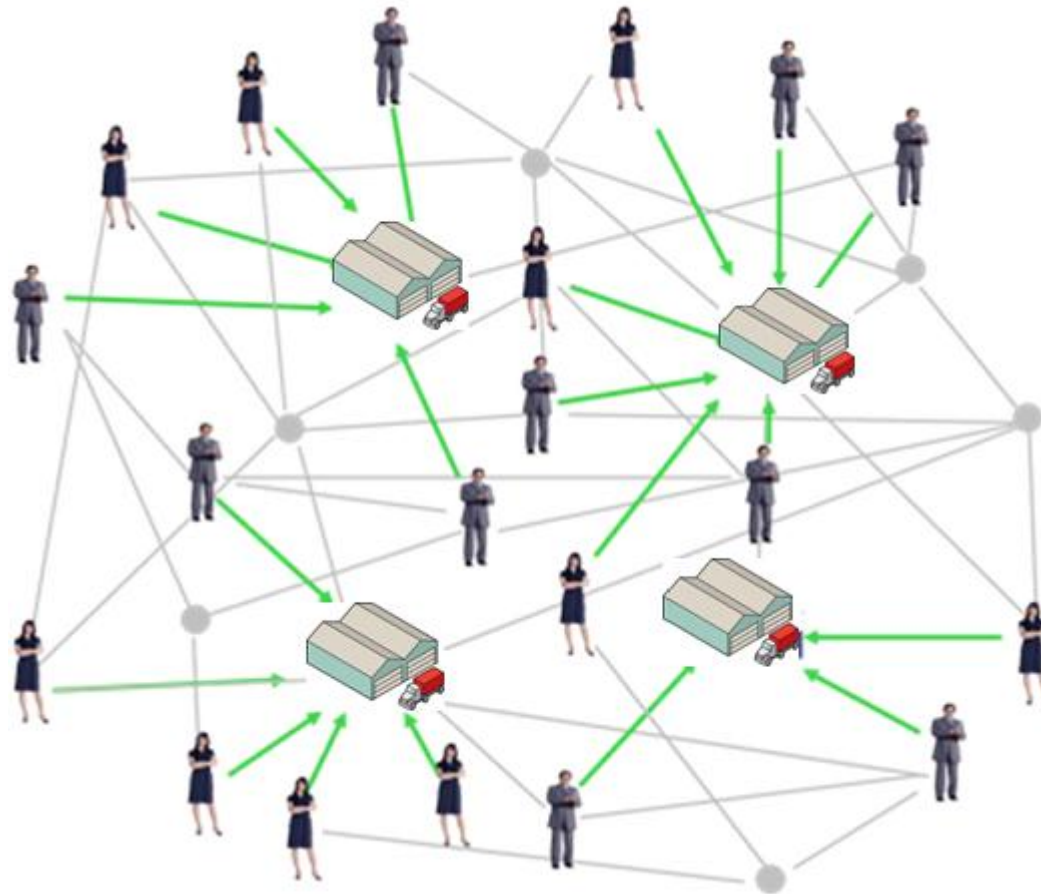
- Almost all solutions were sound (pay attention to quadratic formulations)
- LP are extremely fast to solve in practice
- Difficulty in solving IPs
- Integrality gap can be quite high
- Valid inequalities to create tighter constraints
- Still requires considerable effort to solve IPs to optimality
- How would you approach building an IP Solver?

Project III



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

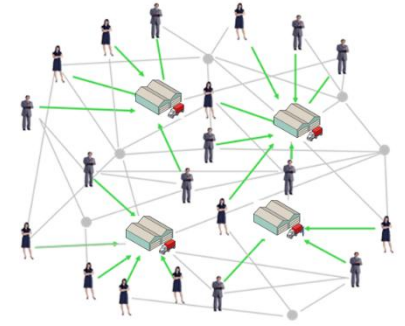
openingCost: 1.0 1.0 1.0 1.0 1.0

facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

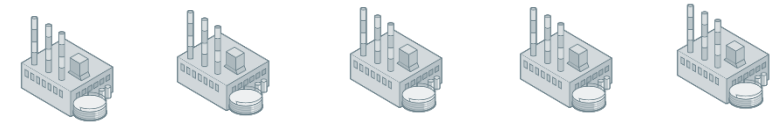
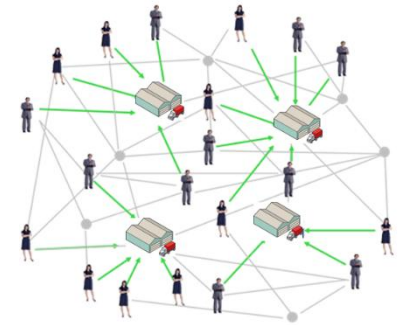
openingCost: 1.0 1.0 1.0 1.0 1.0

facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

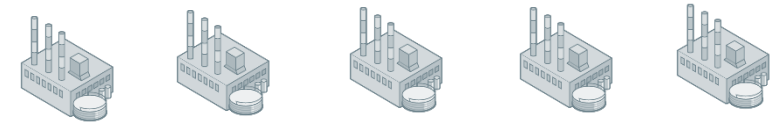
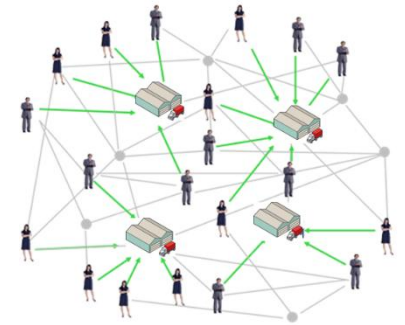
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- Demand is 1, capacity is 2
- How many factory needed at min?



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

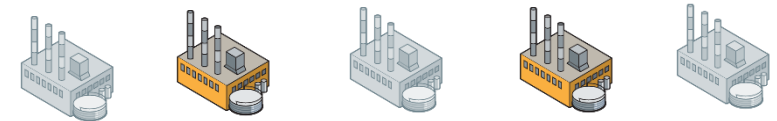
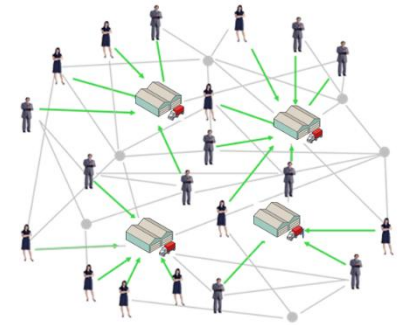
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- Demand is 1, capacity is 2
- How many factory needed at min? 2



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

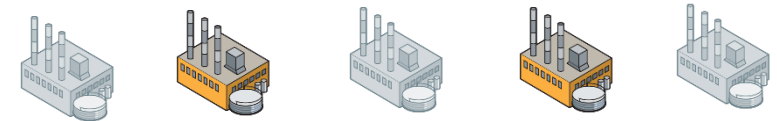
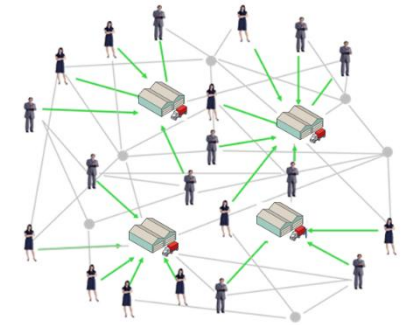
openingCost: 1.0 1.0 1.0 1.0 1.0

facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0



- Can one truck serve 1+ customers?
- How many trucks needed?

Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

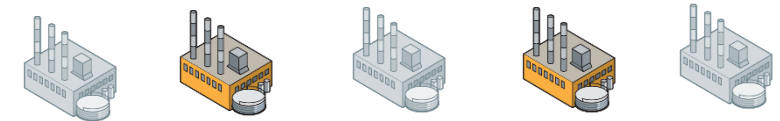
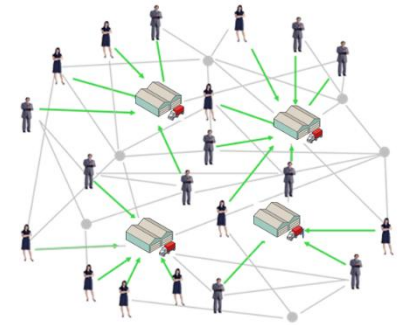
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- Can one truck serve 1+ customers?
- How many trucks needed? 3



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

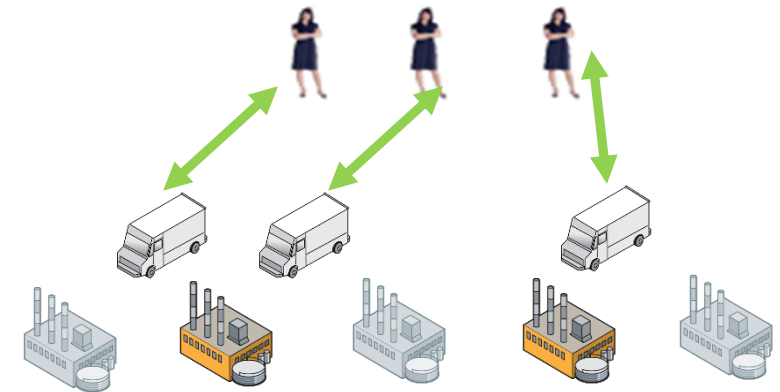
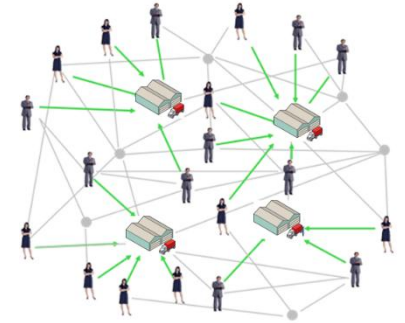
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- Can one truck serve 1+ customers?
- How many trucks needed? 3



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

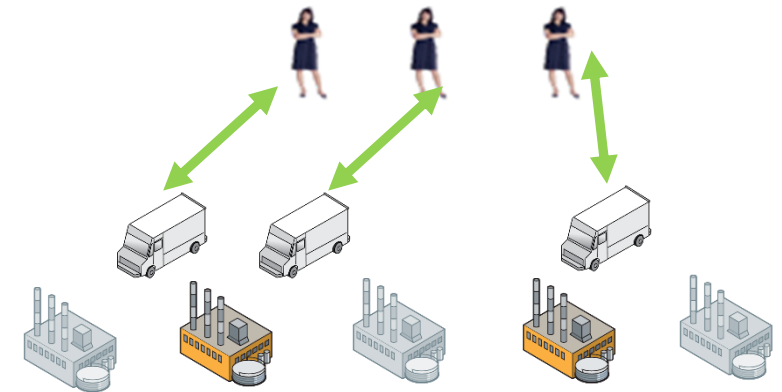
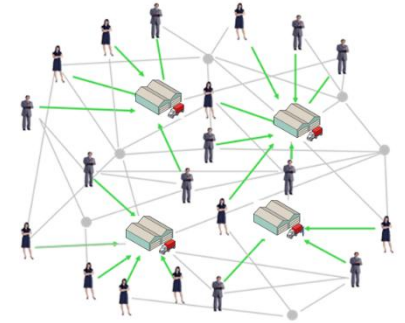
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

➤ What is the cost?



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

facilityCap: 2.0 2.0 2.0 2.0 2.0

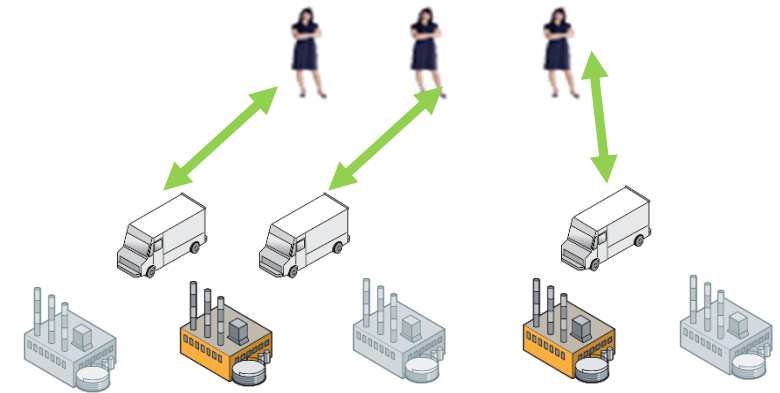
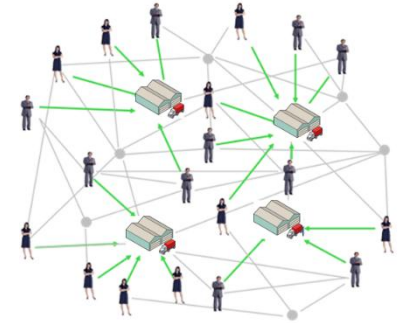
truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

➤ What is the cost?

➤ Opening cost: $1 + 1 = 2$



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

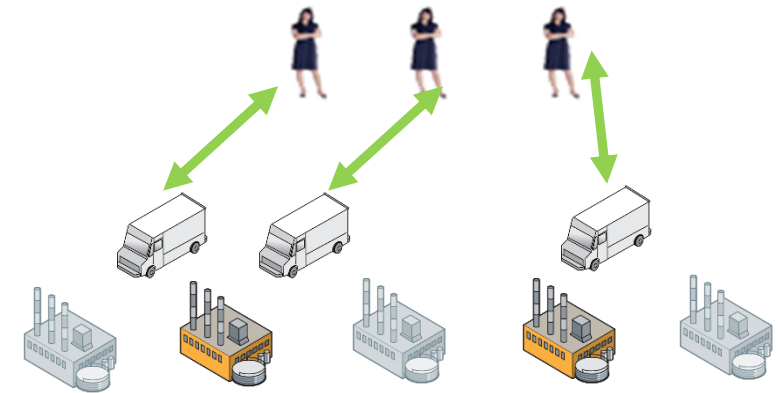
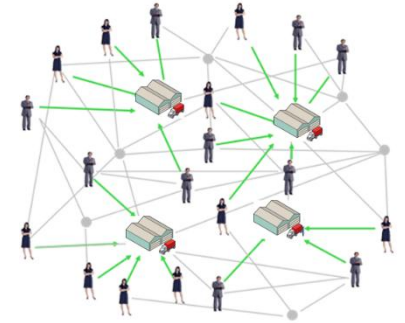
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- What is the cost? 2 +
- Service/Allocation cost: $1 + 1 + 1 = 3$



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

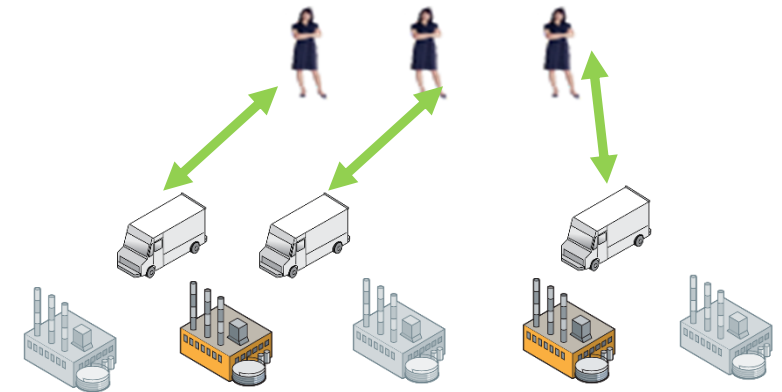
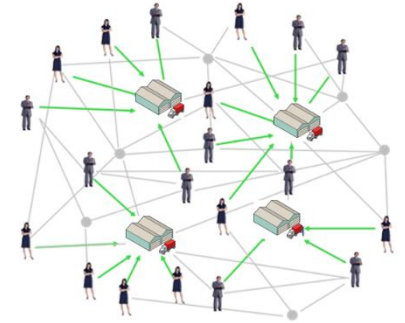
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 9.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- What is the cost? $2 + 3 + 60 = 65$
- Fixed truck usage cost: $20 + 20 + 20 = 60$



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

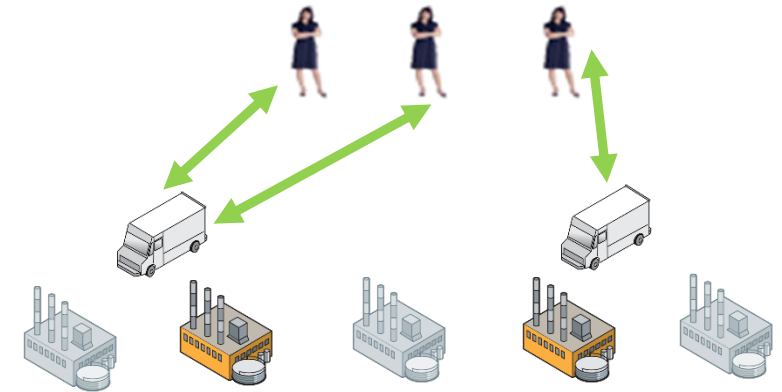
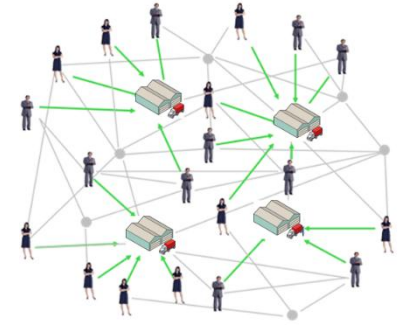
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 10.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- What is the cost? $2 + 3 + 40 = 45$
- Fixed truck usage cost: $20 + 20 = 40$



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

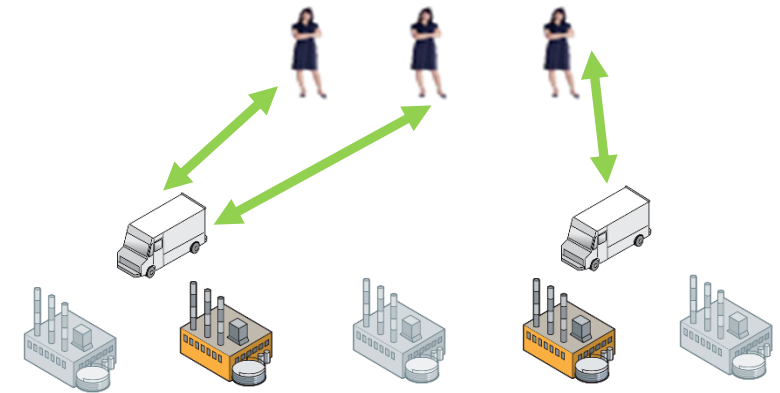
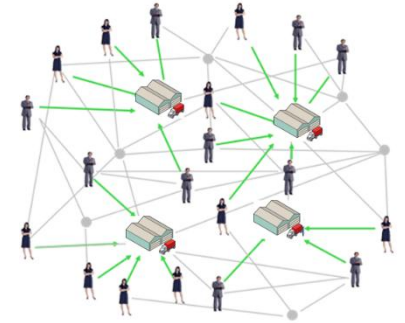
facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 10.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- If you solve your model with integer variables
- You should get the same cost (65 or 45)



Project – III: Supply-Chain Management

Find a lower bound on the cost using LP

numCustomers: 3

numFacilities: 5

numVehicles: 3

Allocation Cost: 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0 1.0

customerDemand: 1.0 1.0 1.0

openingCost: 1.0 1.0 1.0 1.0 1.0

facilityCap: 2.0 2.0 2.0 2.0 2.0

truckDistLimit: 10.0

truckUsageCost: 20.0

travelDistance: 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0 5.0

- We are interested in a lower bound on the cost
- See how close is the estimate?

